Rede Adaline

Disciplina: Redes Neurais Artificiais

Prof. Dr. Lucas C. Ribas

Programa de Pós-Graduação em Ciência da Computação
Departamento de Ciências de Computação e Estatística
Universidade Estadual Paulista (UNESP)

Sumário

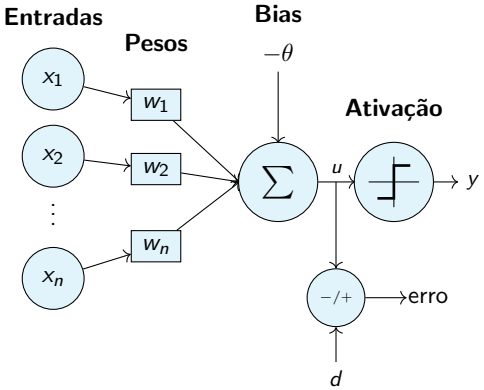
1. ADALINE
2. Regra Delta
3. Implementação Computacional

ADALINE

Aspectos históricos do ADALINE

- O **ADALINE** (*Adaptive Linear Element*) foi idealizado por Widrow & Hoff em 1960.
- Sua principal aplicação estava em sistemas de chaveamento de circuitos telefônicos.
- Esta foi uma das primeiras aplicações industriais que envolveram efetivamente a utilização das redes neurais artificiais.
- Embora seja também um tipo de rede bem simples (um único neurônio), o **ADALINE** promoveu alguns outros avanços para a área de redes neurais, isto é:
  - Desenvolvimento do algoritmo de aprendizado **regra Delta**.
  - Aplicações em diversos problemas práticos envolvendo processamento de sinais analógicos.
  - Primeiras aplicações industriais de redes neurais artificiais.

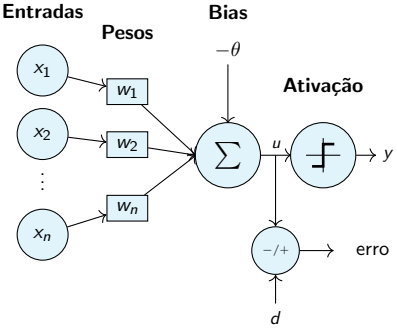
Ilustração da rede ADALINE:



- **Sinais de entrada** $\Rightarrow \{x_1, x_2, \ldots, x_n\}$
- **Pesos sinápticos** $\Rightarrow \{w_1, w_2, \ldots, w_n\}$
- **Combinador linear** $\Rightarrow \sum$
- **Limiar de ativação** $\Rightarrow \{\theta\}$
- **Potencial de ativação** $\Rightarrow \{u\}$
- **Função de ativação** $\Rightarrow \{g\}$
- **Sinal de saída** $\Rightarrow \{y\}$
- **Valor desejado** $\Rightarrow \{d\}$
- **Valor de erro** $\Rightarrow \{erro\}$ entre $\{d\}$ e $\{u\}$

1. Rede **ADALINE** é constituída de n sinais de entrada e somente uma saída
2. Como o **Perceptron**, a rede **ADALINE** possui também arquitetura *feedforward* de camada única.
3. Como o **Perceptron**, é utilizada para classificação envolvendo apenas duas classes distintas.

Princípio de Funcionamento

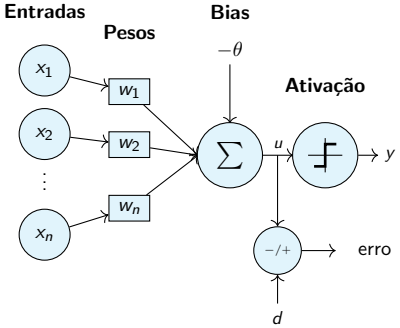


$$u = \sum_{i=1}^{n} w_i \cdot x_i - \theta = \sum_{i=0}^{n} w_i \cdot x_i$$

$$y = g(u)$$

$$erro = d - u$$

Princípio de Funcionamento



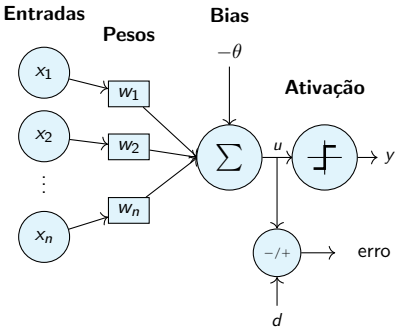
$$u = \sum_{i=1}^{n} w_i \cdot x_i - \theta = \sum_{i=0}^{n} w_i \cdot x_i$$

$$y = g(u)$$

$$erro = d - u$$

1. Apresentação de um conjunto de valores que representam as variáveis de entrada do neurônio.
2. Multiplicação de cada entrada pelo seu respectivo peso sináptico.
3. Obtenção do potencial de ativação somando os sinais ponderados e subtraindo o limiar.
4. Aplicação de uma função de ativação adequada para limitar a saída.
5. Cálculo da saída com base no potencial de ativação e função de ativação.
6. *Bloco associador*, cuja função é produzir o sinal de "erro" entre "d" e "u" para auxiliar no treinamento.

Princípio de Funcionamento

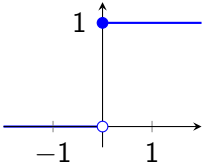


- Como no **Perceptron**, devido às suas características estruturais, as funções de ativação são a **"degrau"** ou **"degrau bipolar"**.
- Assim, tem-se também apenas **duas possibilidades** de valores a serem produzidos pela sua saída:

$$u = \sum_{i=1}^{n} w_i x_i - \theta = \sum_{i=0}^{n} w_i x_i \quad \Rightarrow \quad y = g(u)$$

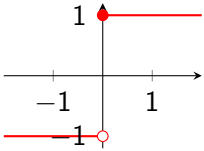
Função Degrau

$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ 0, & \text{se } u < 0 \end{cases}$$

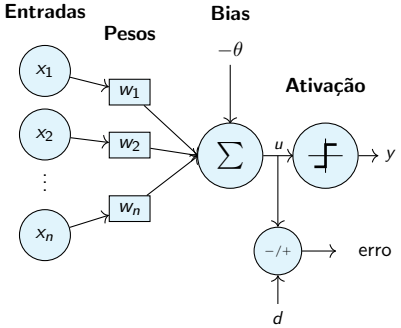


Degrau Bipolar

$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ -1, & \text{se } u < 0 \end{cases}$$



Princípio de Funcionamento



Parâmetro	Variável Representativa	Tipo Característico
Entradas	x_i (*i-ésima entrada*)	Reais ou Binária
Pesos Sinápticos	w_i (associado a x_i)	Reais (iniciados aleatoriamente)
Limiar	θ	Real (iniciado aleatoriamente)
Saída	y	Binária
Função de Ativação	$g(.)$	Degrau ou Degrau Bipolar
Processo de Treinamento	——	Supervisionado
Regra de Aprendizado	——	Regra Delta

Processo de treinamento supervisionado

Parâmetro	Variável Representativa	Tipo Característico
Entradas	x_i (*i-ésima entrada*)	Reais ou Binária
Pesos Sinápticos	w_i (associado a x_i)	Reais (iniciados aleatoriamente)
Limiar	θ	Real (iniciado aleatoriamente)
Saída	y	Binária
Função de Ativação	$g(.)$	Degrau ou Degrau Bipolar
Processo de Treinamento	——	Supervisionado
Regra de Aprendizado	——	Regra Delta

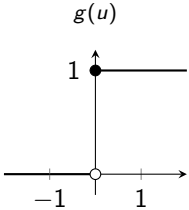
1. Assim como no *Perceptron*, o ajuste dos pesos e limiar do **ADALINE** é efetuado utilizando processo de treinamento "Supervisionado".
2. Então, para cada amostra dos sinais de entrada se tem a respectiva saída (resposta) desejada.
3. Como o **ADALINE** é tipicamente usado em problemas de classificação, a sua saída pode assumir somente dois valores possíveis, os quais se associam às **"duas classes"** que o mesmo estará identificando.
4. A diferença principal entre o **ADALINE** e o *Perceptron* está principalmente na regra de aprendizado utilizada para os ajustes dos pesos e limiar.

Aspectos de Aplicabilidade

1. Para problemas de classificação, assume-se duas classes possíveis: **Classe A** e **Classe B**.
2. A saída do ADALINE é associada a cada uma das classes.

Degrau

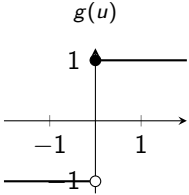
$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ 0, & \text{se } u < 0 \end{cases}$$



$$\text{g(u)} = \begin{cases} 1, & \text{se } u \geq 0 \Rightarrow x \in \text{Classe A} \\ 0, & \text{se } u < 0 \Rightarrow x \in \text{Classe B} \end{cases}$$

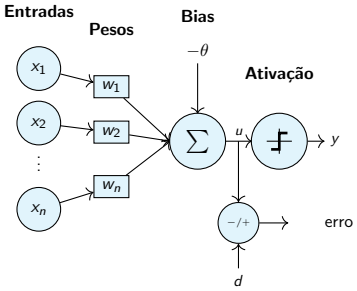
Degrau Bipolar

$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ -1, & \text{se } u < 0 \end{cases}$$



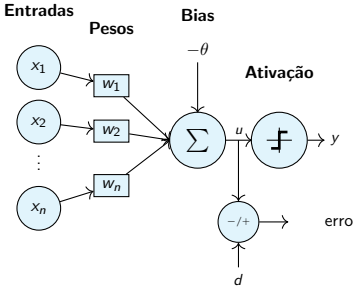
$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \Rightarrow x \in \text{Classe A} \\ -1, & \text{se } u < 0 \Rightarrow x \in \text{Classe B} \end{cases}$$

Análise matemática: O que acontece internamente?



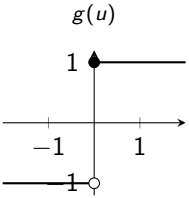
$$u = \sum_{i=1}^{n=2} w_i \cdot x_i - \theta \Rightarrow w_1 \cdot x_1 + w_2 \cdot x_2 - \theta$$

Análise matemática: O que acontece internamente?



$$u = \sum_{i=1}^{n=2} w_i \cdot x_i - \theta \Rightarrow w_1 \cdot x_1 + w_2 \cdot x_2 - \theta$$

- Usando como ativação a função sinal, a saída do *ADALINE* será dada por:



$$y = g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ -1, & \text{se } u < 0 \end{cases}$$

$$\boxed{\mathrm{y} = \mathrm{g(u)} = \begin{cases} 1, & \text{se } u \geq 0 \Leftrightarrow w_1 x_1 + w_2 x_2 - \theta \geq 0 \\ -1, & \text{se } u < 0 \Leftrightarrow w_1 x_1 + w_2 x_2 - \theta < 0 \end{cases}}$$

Classificação de padrões usando o ADALINE

- Assumindo-se aqui um ADALINE com apenas duas entradas, tem-se a fronteira de decisão dada por uma reta cuja equação é definida por:

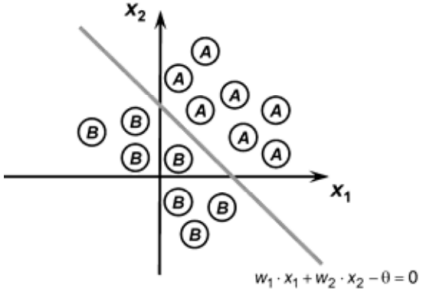
$$w_1 \cdot x_1 + w_2 \cdot x_2 - \theta = 0$$

Classificação de padrões usando o ADALINE

- Assumindo-se aqui um ADALINE com apenas duas entradas, tem-se a fronteira de decisão dada por uma reta cuja equação é definida por:

$$w_1 \cdot x_1 + w_2 \cdot x_2 - \theta = 0$$

Representação gráfica da saída do perceptron



(*)

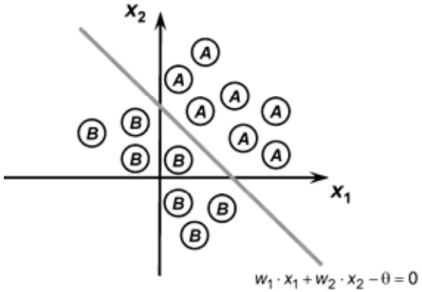
$$\boxed{g(u) = \begin{cases} 1, & \text{se } u \geq 0 \Rightarrow x \in \text{Classe A} \\ -1, & \text{se } u < 0 \Rightarrow x \in \text{Classe B} \end{cases}}$$

Classificação de padrões usando o ADALINE

- Assumindo-se aqui um ADALINE com apenas duas entradas, tem-se a fronteira de decisão dada por uma reta cuja equação é definida por:

$$w_1 \cdot x_1 + w_2 \cdot x_2 - \theta = 0$$

Representação gráfica da saída do perceptron



(*)

$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \Rightarrow x \in \text{Classe A} \\ -1, & \text{se } u < 0 \Rightarrow x \in \text{Classe B} \end{cases}$$

Conclusão: Assim como o **Perceptron**, o **ADALINE** também é um classificador linear que pode ser usado para classificar duas classes **linearmente separáveis**.

Regra Delta

Treinamento Supervisionado

Conceito:

- Usa-se pares entrada-saída desejada para ensinar a rede.
- A rede ajusta seus pesos com base no erro entre saída real e desejada.
- Comporta-se como se houvesse um "professor".

Passos do Treinamento:

1. Apresente uma amostra de treino.
2. Calcule a saída da rede.
3. Compare com a saída desejada.
4. Se tiver dentro de valores aceitáveis, termine o treinamento. Caso contrário, ajuste os pesos.

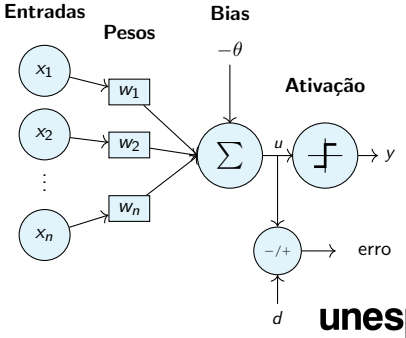
Princípio da regra Delta

- O processo de ajuste dos pesos e limiar do **ADALINE** é baseado no algoritmo de aprendizado denominado **regra Delta**, também conhecida pelos seguintes nomes:
  - ✓ Regra de aprendizado de Widrow-Hoff.
  - ✓ Algoritmo *LMS (Least Mean Square)*.
  - ✓ Regra de atualização obtida pelo método do gradiente descendente.
- Assumindo-se a disponibilidade de p amostras de treinamento, o princípio envolvido com a aplicação da regra **Delta**, objetivando ajustar os pesos e limiar do neurônio, é o seguinte:

✓ **Minimizar a diferença (erro)** entre a saída desejada d e o **potencial de ativação** u (ou saída do combinador linear), considerando todas as p amostras.

✓ Mais especificamente, utiliza-se a minimização do **Erro Quadrático** entre u e d com o intuito de ajustar o vetor de pesos $\{w\}$ da rede.

$$\mathbf{w} = \begin{bmatrix} \theta & w_1 & w_2 & \dots & w_n \end{bmatrix}^T$$



Princípio da regra Delta

- **No Perceptron:**
  - A função de ativação é a **degrau**:

$$g(u) = \begin{cases} 1, & \text{se } u \geq 0 \\ 0, & \text{se } u < 0 \end{cases}$$

  - Define a **classe** diretamente com base em uma fronteira de decisão.
  - O erro para treinamento é calculado com base na saída **após a ativação** ($y \in \{0, 1\}$).
- **No ADALINE:**
  - Erro é contínuo, o que permite aplicar descida do gradiente com Erro Quadrático Médio.
  - O erro é calculado com base no valor **contínuo** de u (função identidade), antes de qualquer limiarização.
  - Apenas na fase de **inferência** (predição), usa-se o **degrau** para rotular as classes.

Derivação do algoritmo regra Delta

Objetivo do algoritmo:

- Consiste em obter um $\mathbf{w}^*$ ótimo tal que o **Erro Quadrático** $E(\mathbf{w}^*)$ sobre todo o conjunto de amostras seja o mínimo possível, isto é:

$$E(\mathbf{w}^*) \leq E(\mathbf{w}), \quad \text{para todo } \mathbf{w} \in \mathbb{R}^{n+1}$$

- A função **Erro Quadrático** em relação às p amostras de treinamento é definida por:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{p}(d^{(k)} - u)^2$$

com $\frac{1}{2}$ e não $\frac{1}{p}$ (não é médio) para fins de simplificação da derivada.

- Usando o valor de u na expressão acima, obtém-se:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{p}\left(d^{(k)} - \left(\sum_{i=1}^{n} w_i \cdot x_i^{(k)} - \theta\right)\right)^2$$

$$\boxed{\mathrm{u} = \textstyle\sum_{i=1}^{n=2} w_i \cdot x_i - \theta}$$

- Em forma vetorial, tem-se:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{p}\left(d^{(k)} - \left(\mathbf{w}^T \cdot \mathbf{x}^{(k)} - \theta\right)\right)^2$$

unesp

Derivação do algoritmo regra Delta

Objetivo do algoritmo:

- Consiste em obter um $\mathbf{w}^*$ ótimo tal que o **Erro Quadrático** $E(\mathbf{w}^*)$ sobre todo o conjunto de amostras seja o mínimo possível, isto é:

$$E(\mathbf{w}^*) \leq E(\mathbf{w}), \quad \text{para todo } \mathbf{w} \in \mathbb{R}^{n+1}$$

- A função **Erro Quadrático** em relação às p amostras de treinamento é definida por:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{p}(d^{(k)} - u)^2$$

com $\frac{1}{2}$ e não $\frac{1}{p}$ (não é médio) para fins de simplificação da derivada.

- Usando o valor de u na expressão acima, obtém-se:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{p}\left(d^{(k)} - \left(\sum_{i=1}^{n} w_i \cdot x_i^{(k)} - \theta\right)\right)^2$$

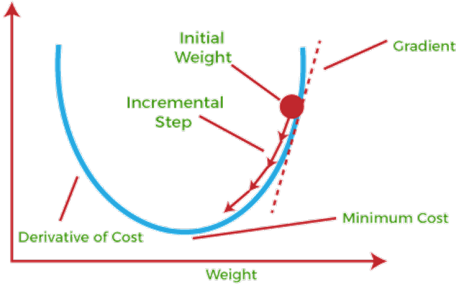
$$\boxed{\mathrm{u} = \textstyle\sum_{i=1}^{n=2} w_i \cdot x_i - \theta}$$

Conclusão: Esta expressão ao lado totaliza o erro quadrático contabilizando-se as p amostras de treinamento.

- Em forma vetorial, tem-se:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{p}\left(d^{(k)} - \left(\mathbf{w}^T \cdot \mathbf{x}^{(k)} - \theta\right)\right)^2$$

Função de Erro Quadrático



Função de Erro no ADALINE:

$$E(w) = \frac{1}{2}\sum_{k=1}^{p}(d^{(k)} - u^{(k)})^2$$

- É uma **função quadrática** em relação aos pesos w.
- Isso significa que seu gráfico tem o formato de um **parábola convexa**.
- Como é **convexa**, possui um único **mínimo global**.
- O gradiente é um vetor que aponta na direção de maior aumento da função.
- A descida do gradiente busca a convergência para o ponto mínimo.

Derivação do algoritmo regra Delta

- **Aplicação do operador Gradiente:** Consiste em aplicar o **operador gradiente** em relação ao vetor $\mathbf{w}$ para analisar como o erro varia com os pesos. Como ele aponta na direção de maior aumento do erro, sua direção oposta indica como minimizá-lo.

$$\nabla E(\mathbf{w}) = \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} \quad \text{onde} \quad E(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{p}\left(d^{(k)} - \left(\mathbf{w}^T \cdot \mathbf{x}^{(k)} - \theta\right)\right)^2$$

- Considerando o valor de $E(\mathbf{w})$ na expressão acima, obtém-se:

$$\boxed{\mathrm{u} = \sum_{i=1}^{n=2} w_i \cdot x_i - \theta}$$

$$\nabla E(\mathbf{w}) = \sum_{k=1}^{p}\left(d^{(k)} - (\mathbf{w}^T \cdot \mathbf{x}^{(k)} - \theta)\right) \cdot (-\mathbf{x}^{(k)})$$

- Retornando o valor de u na expressão acima, tem-se:

$$\nabla E(\mathbf{w}) = -\sum_{k=1}^{p}(d^{(k)} - u) \cdot \mathbf{x}^{(k)}$$

Derivação do algoritmo regra Delta

- **Aplicação do operador Gradiente:** Consiste em aplicar o **operador gradiente** em relação ao vetor $\mathbf{w}$ para analisar como o erro varia com os pesos. Como ele aponta na direção de maior aumento do erro, sua direção oposta indica como minimizá-lo.

$$\nabla E(\mathbf{w}) = \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} \quad \text{onde} \quad E(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{p}\left(d^{(k)} - \left(\mathbf{w}^T \cdot \mathbf{x}^{(k)} - \theta\right)\right)^2$$

- Considerando o valor de $E(\mathbf{w})$ na expressão acima, obtém-se:

$$\nabla E(\mathbf{w}) = \sum_{k=1}^{p}\left(d^{(k)} - (\mathbf{w}^T \cdot \mathbf{x}^{(k)} - \theta)\right) \cdot (-\mathbf{x}^{(k)})$$

- Retornando o valor de u na expressão acima, tem-se:

$$\nabla E(\mathbf{w}) = -\sum_{k=1}^{p}(d^{(k)} - u) \cdot \mathbf{x}^{(k)}$$

$$\boxed{\mathrm{u} = \textstyle\sum_{i=1}^{n=2} w_i \cdot x_i - \theta}$$

Conclusão: Esta expressão ao lado fornece o valor do gradiente do erro em relação a todas as amostras de treinamento.

unesp

Ajuste do vetor de pesos

- Agora, deve-se associar o valor do gradiente aos procedimentos de ajuste dos pesos $\{\mathbf{w}\}$. Assim, a adaptação do vetor de pesos deve ser efetuada na direção oposta àquela do gradiente (fator -1), pois o objetivo da otimização é minimizar o erro quadrático médio entre d e u.
- Nesta condição, a variação do peso $\Delta\mathbf{w}$ a ser aplicado no vetor de pesos do ADALINE é dada por:

$$\Delta\mathbf{w} = -\eta \cdot \nabla E(\mathbf{w}) \quad \text{onde} \quad \nabla E(\mathbf{w}) = -\sum_{k=1}^{p} (d^{(k)} - u) \cdot \mathbf{x}^{(k)}$$

- Inserindo o resultado de $E(\mathbf{w})$ na expressão acima, tem-se:

$$\Delta\mathbf{w} = \eta \sum_{k=1}^{p} (d^{(k)} - u) \cdot \mathbf{x}^{(k)}$$

- De forma complementar, pode-se exprimir a variação $\Delta\mathbf{w}$ e a atualização dos pesos por:

$$\mathbf{w}^{\text{atual}} = \mathbf{w}^{\text{anterior}} + \eta \sum_{k=1}^{p} (d^{(k)} - u) \cdot \mathbf{x}^{(k)}$$

Simplificação algorítmica

- Por simplicidade computacional, atualização de $\mathbf{w}$ pode ser também feita de forma discreta.
- Neste caso, um passo de ajuste em $\mathbf{w}$ é realizado após a apresentação de cada k-ésima amostra de treinamento (SGD) e não acumulando todas (Batch GD), ou seja:

$$\mathbf{w}^{\text{atual}} = \mathbf{w}^{\text{anterior}} + \eta \sum_{k=1}^{p} (d^{(k)} - u) \cdot \mathbf{x}^{(k)}$$

$$\mathbf{w}^{\text{atual}} = \mathbf{w}^{\text{anterior}} + \eta \cdot (d^{(k)} - u) \cdot \mathbf{x}^{(k)}, \quad k = 1 \ldots p$$

Simplificação algorítmica

- Por simplicidade computacional, atualização de $\mathbf{w}$ pode ser também feita de forma discreta.
- Neste caso, um passo de ajuste em $\mathbf{w}$ é realizado após a apresentação de cada k-ésima amostra de treinamento (SGD) e não acumulando todas (Batch GD), ou seja:

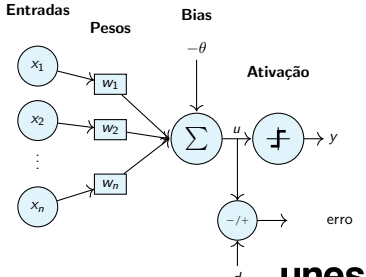
$$\mathbf{w}^{\text{atual}} = \mathbf{w}^{\text{anterior}} + \eta \sum_{k=1}^{p} (d^{(k)} - u) \cdot \mathbf{x}^{(k)}$$

$$\mathbf{w}^{\text{atual}} = \mathbf{w}^{\text{anterior}} + \eta \cdot (d^{(k)} - u) \cdot \mathbf{x}^{(k)}, \quad k = 1 \ldots p$$

- Em notação algorítmica:

$$\boxed{\mathbf{w} \leftarrow \mathbf{w} + \eta \cdot (d^{(k)} - u) \cdot \mathbf{x}^{(k)}, \quad k = 1 \ldots p}$$

  - $\mathbf{w} = [\theta \; w_1 \; w_2 \ldots w_n]^T$
  - $\mathbf{x}^{(k)} = [-1 \; x_1^{(k)} \; x_2^{(k)} \ldots x_n^{(k)}]^T$
  - $d^{(k)}$ é a saída desejada
  - $\eta \in (0, 1)$ é a taxa de aprendizado



Batch vs Stochastic Gradient Descent no ADALINE

- Ambos os métodos visam minimizar a função de erro quadrático (MSE), que é convexa.
- A diferença está na forma como os pesos $\mathbf{w}$ são atualizados:

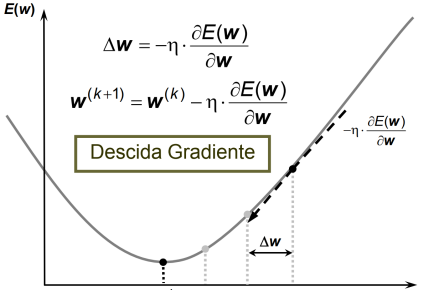
Batch Gradient Descent	**Stochastic Gradient Descent (SGD)**
Atualiza os pesos após processar **todas as amostras**	Atualiza os pesos **após cada amostra**
Mais estável e suave	Mais rápido, porém com oscilações
Convergência mais lenta, porém direta	Convergência mais rápida, porém ruidosa
Requer mais memória (usa todo o dataset)	Requer menos memória
Ideal para pequenos ou médios datasets*	Ideal para grandes datasets ou treinamento on-line *

Conclusão: Ambos podem encontrar o mesmo $\mathbf{w}^*$, mas o caminho será diferente.

Interpretação geométrica: processo de convergência do ADALINE

A interpretação geométrica frente aos passos de atualização de $\mathbf{w}$, rumo ao ponto de minimização $\mathbf{w}^*$ da função erro quadrático $E(\mathbf{w})$, é dada pela seguinte ilustração:

1. Parte-se de um valor inicial $\mathbf{w}^{(0)}$;
2. O próximo valor de $\mathbf{w}$ (dado por $\mathbf{w}^{(1)}$) será obtido considerando-se a direção oposta ao vetor gradiente em relação a $\mathbf{w}^{(0)}$;
3. No próxima atualização, o ajuste de $\mathbf{w}$ (representado agora por $\mathbf{w}^{(2)}$) será realizado considerando-se o valor do gradiente em relação à $\mathbf{w}^{(1)}$;
4. Aplicando-se tais passos sucessivamente, haverá a convergência em direção a $\mathbf{w}^*$;
5. Este valor de $\mathbf{w}^*$ será a configuração ótima para os parâmetros livres do *ADALINE*;
6. O valor de $E(\mathbf{w}^*)$ será então sempre menor que quaisquer $E(\mathbf{w})$ calculados nos passos anteriores.



(*)

Critério de parada do algoritmo

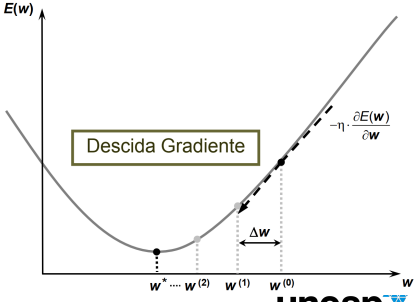
- Com a aplicação dos passos iterativos anteriores, o vetor $\mathbf{w}$ caminhará em direção ao seu valor ótimo $\{\mathbf{w}^*\}$.
- Para se detectar o alcance de $\mathbf{w}^*$, a fim de cessar o processo de convergência, há a necessidade de se estabelecer um critério de parada.
- Para tanto, utiliza-se o valor do erro quadrático médio $\{E_{qm}(\mathbf{w})\}$ em relação a todas as amostras de treinamento, sendo este definido por:

$$E_{qm}(\mathbf{w}) = \frac{1}{p}\sum_{k=1}^{p}(d^{(k)} - u)^2$$

- O algoritmo converge quando o erro quadrático médio entre duas épocas sucessivas for bem pequeno, isto é:

$$\left| E_{qm}(\mathbf{w}^{\text{atual}}) - E_{qm}(\mathbf{w}^{\text{anterior}}) \right| \leq \varepsilon$$

onde ε é a precisão requerida para o processo de convergência.



(*)

Implementação Computacional

Implementação Computacional

Para montagem de conjuntos de treinamento, suponha:

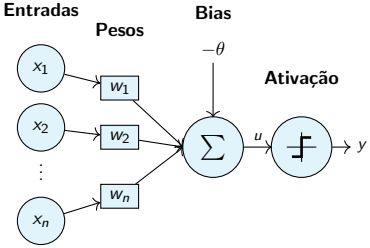
- Um problema a ser mapeado pelo **ADALINE** com três entradas $\{x_1, x_2, x_3\}$.
- Quatro amostras com os seguintes valores de entrada e saídas desejadas:

Amostra 1: Entrada: $[0.1,\ 0.4,\ 0.7] \Rightarrow$ Saída desejada: $[1]$

Amostra 2: Entrada: $[0.3,\ 0.7,\ 0.2] \Rightarrow$ Saída desejada: $[-1]$

Amostra 3: Entrada: $[0.6,\ 0.9,\ 0.8] \Rightarrow$ Saída desejada: $[-1]$

Amostra 4: Entrada: $[0.5,\ 0.7,\ 0.1] \Rightarrow$ Saída desejada: $[1]$



Implementação Computacional

- Então, de forma similar ao **Perceptron**, pode-se converter tais sinais para que os mesmos possam ser usados no treinamento do **ADALINE**:

Conjunto de Treinamento:

$$\mathbf{x}^{(1)} = [-1,\ 0.1,\ 0.4,\ 0.7]^T \Rightarrow d^{(1)} = 1$$

$$\mathbf{x}^{(2)} = [-1,\ 0.3,\ 0.7,\ 0.2]^T \Rightarrow d^{(2)} = -1$$

$$\mathbf{x}^{(3)} = [-1,\ 0.6,\ 0.9,\ 0.8]^T \Rightarrow d^{(3)} = -1$$

$$\mathbf{x}^{(4)} = [-1,\ 0.5,\ 0.7,\ 0.1]^T \Rightarrow d^{(4)} = 1$$

$$\Omega^{(\boldsymbol{x})} = \begin{array}{c} \\ x_0 \\ x_1 \\ x_2 \\ x_3 \end{array} \begin{array}{c} \begin{array}{cccc} \boldsymbol{x}^{(1)} & \boldsymbol{x}^{(2)} & \boldsymbol{x}^{(3)} & \boldsymbol{x}^{(4)} \end{array} \\ \begin{bmatrix} -1 & -1 & -1 & -1 \\ 0{,}1 & 0{,}3 & 0{,}6 & 0{,}5 \\ 0{,}4 & 0{,}7 & 0{,}9 & 0{,}7 \\ 0{,}7 & 0{,}2 & 0{,}8 & 0{,}1 \end{bmatrix} \end{array}$$

$$\Omega^{(\boldsymbol{d})} = \begin{array}{c} \begin{array}{cccc} \boldsymbol{d}^{(1)} & \boldsymbol{d}^{(2)} & \boldsymbol{d}^{(3)} & \boldsymbol{d}^{(4)} \end{array} \\ \begin{bmatrix} 1 & -1 & -1 & 1 \end{bmatrix} \end{array}$$

(*).

- **Forma Matricial:** as amostras de treinamento são organizadas como colunas em uma matriz:

Pseudocódigo para fase de treinamento

Início {Algoritmo *Adaline* – Fase de Treinamento}

<1> Obter o conjunto de amostras de treinamento $\{\boldsymbol{x}^{(k)}\}$;

<2> Associar a saída desejada $\{d^{(k)}\}$ para cada amostra obtida;

<3> Iniciar o vetor $\boldsymbol{w}$ com valores aleatórios pequenos;

<4> Especificar taxa de aprendizagem $\{\eta\}$ e precisão requerida $\{\varepsilon\}$;

<5> Iniciar o contador de número de épocas { *época* $\leftarrow 0$ };

<6> Repetir as instruções:

<6.1> $E_{qm}^{anterior} \leftarrow E_{qm}(\boldsymbol{w})$;

<6.2> Para todas as amostras de treinamento $\{\boldsymbol{x}^{(k)},\ d^{(k)}\}$, fazer:

<6.2.1> $u \leftarrow \boldsymbol{w}^T \cdot \boldsymbol{x}^{(k)}$;

<6.2.2> $\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta \cdot (d^{(k)} - u) \cdot \boldsymbol{x}^{(k)}$;

<6.3> *época* $\leftarrow$ *época* + 1;

<6.4> $E_{qm}^{atual} \leftarrow E_{qm}(\boldsymbol{w})$;

Até que: $\left| E_{qm}^{atual} - E_{qm}^{anterior} \right| \leq \varepsilon$

(*) **Fim {Algoritmo *Adaline* – Fase de Treinamento}**

Critério de parada: O algoritmo converge quando o erro quadrático médio E_{qm} entre duas épocas sucessivas for menor ou igual à precisão ε requerida ao problema mapeado pelo ADALINE.

Pseudocódigo para fase de teste

Início {Algoritmo *Adaline* – Fase de Operação}

<1> Obter uma amostra a ser classificada { $\boldsymbol{x}$ };

<2> Utilizar o vetor $\boldsymbol{w}$ ajustado durante o treinamento;

<3> Executar as seguintes instruções:

- <3.1> $u \leftarrow \boldsymbol{w}^T \cdot \boldsymbol{x}$;
- <3.2> $y \leftarrow \text{sinal}(u)$;
- <3.3> Se $y = -1$
  - <3.3.1> Então: amostra $\boldsymbol{x} \in$ *{Classe A}*
- <3.4> Se $y = 1$
  - <3.4.1> Então: amostra $\boldsymbol{x} \in$ *{Classe B}*

(*) **Fim {Algoritmo *Adaline* – Fase de Operação}**

Obs 1: A *Fase de Operação ou Teste* é usada somente após a fase de treinamento, pois aqui a rede já está apta para ser usada no processo.

Obs 2: A *Fase de Operação ou Teste* é então utilizada para realizar a tarefa de classificação frente às novas amostras.

Obs 3: Lembrar de incluir o valor -1 dentro do vetor $\mathbf{x}$, pois será multiplicado por $\mathbf{w}$.

$$\mathbf{x} = \begin{bmatrix} -1 & x_1 & x_2 & \cdots & x_n \end{bmatrix}^T$$

$$\mathrm{w} = \begin{bmatrix} \theta & w_1 & w_2 & \cdots & w_n \end{bmatrix}^T$$

unesp

Pseudocódigo para o erro quadrático médio

Início {Algoritmo EQM}

<1> Obter a quantidade de padrões de treinamento { p };

<2> Iniciar a variável E_{qm} com valor zero { $E_{qm} \leftarrow 0$ };

<3> Para todas as amostras de treinamento { $\boldsymbol{x}^{(k)}$, $d^{(k)}$ }, fazer:

<3.1> $u \leftarrow \boldsymbol{w}^T \cdot \boldsymbol{x}^{(k)}$;

<3.2> $E_{qm} \leftarrow E_{qm} + (d^{(k)} - u)^2$;

<4> $E_{qm} \leftarrow \dfrac{E_{qm}}{p}$;

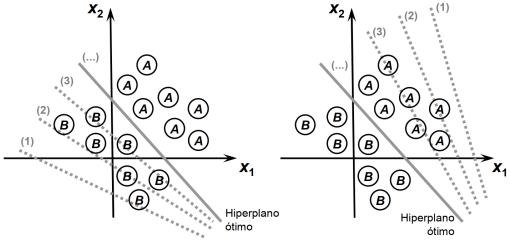
(*) **Fim {Algoritmo EQM}**

Obs 1: O algoritmo do erro quadrático médio deve ser implementado por meio de uma sub-rotina (função).

Obs 2: Todas as amostras de treinamento devem ser contabilizadas no cálculo do valor do erro quadrático médio.

Ilustração do processo de convergência

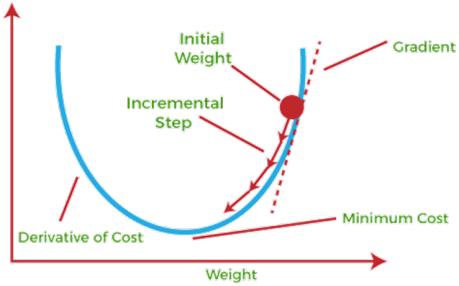
- Processo de treinamento do *ADALINE* tende a mover continuamente o seu vetor de pesos, tendo-se o intuito de minimizar o EQ em relação a todas as amostras de treino
- As figuras mostram duas situações que ilustram a convergência do *ADALINE* rumo à estabilização, considerando-se para fins didáticos apenas duas entradas $\{x_1$ e $x_2\}$.



(*)

- Nas duas situações, os vetores de pesos iniciais $\mathbf{w}^{(0)}$ foram iniciados em **posições distintas**, mas processo de convergência encaminha o hiperplano **sempre para o mesmo local.**
- Nesses casos, o valor correspondente a $\mathbf{w}^*$ é sempre aquele que minimiza a função erro quadrático.

Forma Quadrática e Convexa da Função de Erro no ADALINE



Função de Erro no ADALINE:

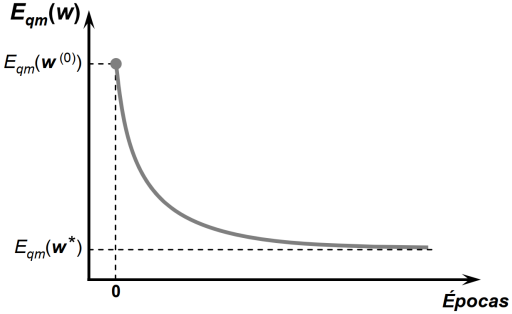
$$E(w) = \frac{1}{2}\sum_{k=1}^{p}(d^{(k)} - u^{(k)})^2$$

- É uma **função quadrática** em relação aos pesos w.
- Isso significa que seu gráfico tem o formato de um **parábola convexa**.
- Como é **convexa**, possui um único **mínimo global**.
- A descida do gradiente garante convergência a esse ponto mínimo, independentemente do ponto de partida.

Conclusão: A forma quadrática e convexa do erro no ADALINE permite que o algoritmo encontre sempre a melhor solução possível.

Comportamento da função erro quadrático

- Para fins de interpretação, a figura mostra o comportamento do erro quadrático médio E_{qm} em função do número de épocas de treinamento.

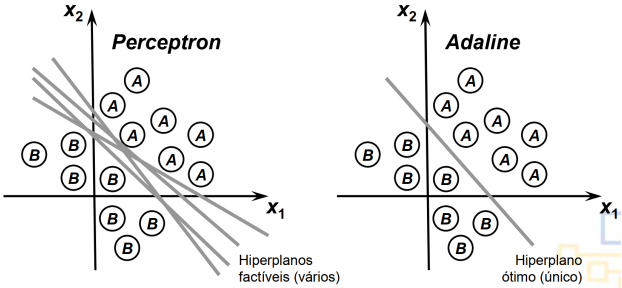


(*)

- Constata-se que a curva do E_{qm} para o *ADALINE* é sempre descendente.
- Esta decresce na medida em que as épocas de treinamento vão sendo executadas.
- Finalmente, estabiliza-se num valor constante quando o ponto de mínimo da função erro quadrático médio for alcançado.

Comparação entre ADALINE e Perceptron

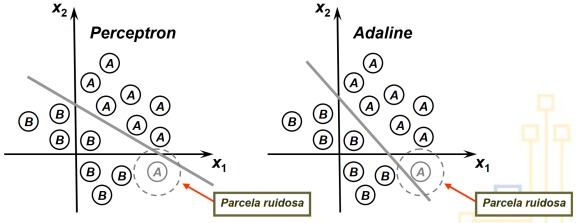
- **ADALINE**: Treinamento usando Regra Delta (Minimização do erro em relação a todas as amostras de treinamento).
  - ◇ Independente dos valores iniciais atribuídos a **w**, o hiperplano de separabilidade (após convergência) sempre será o mesmo.
- **Perceptron**: Treinamento usando Regra de Hebb (Avaliação das respostas produzidas após apresentação de cada amostra de treinamento).
  - ◇ Quaisquer hiperplanos posicionados dentro da faixa de separabilidade entre as classes são considerados soluções factíveis.



(*)

Comparação entre ADALINE e Perceptron

- **ADALINE:** A inclinação do hiperplano é ajustada por intermédio do método dos mínimos quadrados dos erros (processo otimizado).
  - Maior tendência de imunidade a eventuais ruídos que podem afetar o processo em que a mesma está mapeando.
- **Perceptron** O hiperplano que separa as classes pode ter infinitas disposições, pois sua configuração final é fortemente dos valores iniciais de **w**.
  - Menor tendência de imunidade a eventuais ruídos que podem afetar o processo em que a mesma está mapeando.



(*)

- **ADALINE**: probabilidade menor de classificar a amostra incorretamente.
- **Perceptron**: probabilidade maior de classificar a amostra incorretamente.

Exemplo: Execução do ADALINE com 2 amostras

Configuração:

- $\mathbf{w} = [\theta,\ w_1,\ w_2] = [0{,}5,\ 0{,}3,\ 0]$
- $\eta = 0{,}1$
- Amostras: $\mathbf{x}^{(1)} = [-1,\ 1,\ 0], \quad d^{(1)} = 1 \qquad \mathbf{x}^{(2)} = [-1,\ 1,\ 1], \quad d^{(2)} = -1$

Iteração (época 1):

- **Amostra 1:**

$$u = \mathbf{w} \cdot \mathbf{x}^{(1)} = 0{,}5 \cdot (-1) + 0{,}3 \cdot 1 + 0 \cdot 0 = -0{,}2$$

$$erro = d - y = 1 - (-0{,}2) = 1{,}2$$

$$\Delta\mathbf{w} = \eta \cdot erro \cdot \mathbf{x}^{(1)} = 0{,}1 \cdot 1{,}2 \cdot [-1, 1, 0] = [-0{,}12,\ 0{,}12,\ 0]$$

$$\mathbf{w} = [0{,}5,\ 0{,}3,\ 0] + [-0{,}12,\ 0{,}12,\ 0] = [0{,}38,\ 0{,}42,\ 0]$$

- **Amostra 2:**

$$u = \mathbf{w} \cdot \mathbf{x}^{(2)} = 0{,}38 \cdot (-1) + 0{,}42 \cdot 1 + 0 \cdot 1 = 0{,}04$$

$$erro = -1 - 0{,}04 = -1{,}04$$

$$\Delta\mathbf{w} = 0{,}1 \cdot (-1{,}04) \cdot [-1, 1, 1] = [0{,}104,\ -0{,}104,\ -0{,}104]$$

$$\mathbf{w} = [0{,}38,\ 0{,}42,\ 0] + [0{,}104,\ -0{,}104,\ -0{,}104] = [0{,}484,\ 0{,}316,\ -0{,}104]$$

Reflexões, observações e aspectos práticos

1. Explique por que o treinamento do **ADALINE** se processa normalmente de forma mais rápido que aquele do *Perceptron*.

2. Discorra se a afirmação seguinte é verdadeira ou falsa.
   Independente dos valores iniciais assumidos para o vetor de pesos do **ADALINE**, uma mesma configuração final para $\mathbf{w}^*$ será sempre obtida após a sua convergência.

3. Explique, considerando a questão anterior, se o número de épocas de treinamento será também igual, independentemente do seu vetor de pesos iniciais.

4. Considerando-se que um problema a ser mapeado pelo **ADALINE** não seja linearmente separável, explique então se para esta situação o processo de treinamento (por meio do algoritmo regra Delta) também convergirá.

Referência e Agradecimentos

Slides baseados em:

- Notas de aula AULA 04 – Rede ADALINE, disciplina de "Redes Neurais Artificiais". Prof. Ivan Nunes da Silva. (*) Elementos (Figuras e Códigos) retiradas dos slides e do livro abaixo.
- Notas de aula AULA 02 – Introdução as RNAs, disciplina de "Redes Neurais Artificiais". Prof. Ivan Nunes da Silva. (*) Elementos (Figuras e Códigos) retiradas dos slides e do livro abaixo.
- Silva, Ivan Nunes da, Danilo Hernane Spatti, and Rogério Andrade Flauzino. "Redes neurais artificiais para engenharia e ciências aplicadas." (2010).

Regra da Cadeia

É utilizada para derivar funções compostas, ou seja, quando uma função está dentro de outra.

Se $y = f(g(x))$, definindo $u = g(x)$, então:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Derivamos a função mais externa e multiplicamos pela derivada da função interna ("de fora para dentro"). **Exemplo:** $y = \sin(x^2)$, temos $u = x^2 \quad \Rightarrow \quad y = \sin(u)$

- **Passo 1 (função externa):** derivamos $\sin(u)$ mantendo a função interna u:

$$\frac{dy}{du} = \cos(u) \Rightarrow \cos(x^2)$$

- **Passo 2 (função interna):** derivamos $u = x^2$:

$$\frac{du}{dx} = \frac{d}{dx}(x^2) = 2x$$

Resultado final:

$$\frac{dy}{dx} = \cos(x^2) \cdot 2x$$

Derivando o Erro Quadrático — Parte 1

Objetivo: derivar a função de erro em relação ao vetor de pesos $\mathbf{w}$, considerando uma única amostra:

$$E(\mathbf{w}) = \frac{1}{2}(d-u)^2 \qquad \text{com} \qquad u = \mathbf{w}^T\mathbf{x} - \theta$$

- O valor desejado d é uma constante, ou seja, não depende dos pesos.
- Já o potencial de ativação u depende de $\mathbf{w}$, pois é calculado a partir das entradas e dos pesos.
- Como $E(\mathbf{w})$ depende de u, e u depende de $\mathbf{w}$, utilizamos a **regra da cadeia**.
- Definindo:

$$g(\mathbf{w}) = d - u \quad \Rightarrow \quad E(\mathbf{w}) = \frac{1}{2}g(\mathbf{w})^2$$

- Aplicando a regra da cadeia:

$$\frac{\partial E}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}}\left(\frac{1}{2}g(\mathbf{w})^2\right)$$

- Derivando o termo quadrático:

$$\frac{\partial E}{\partial \mathbf{w}} = \frac{1}{2}\cdot 2g(\mathbf{w})\cdot\frac{\partial g}{\partial \mathbf{w}}$$

- Substituindo $g(\mathbf{w}) = d - u$:

$$\frac{\partial E}{\partial \mathbf{w}} = \frac{1}{2}\cdot 2(d-u)\cdot\frac{\partial (d-u)}{\partial \mathbf{w}}$$

Derivando o Erro Quadrático — Parte 2

Agora derivamos o termo interno:

$$\frac{\partial(d-u)}{\partial \mathbf{w}} = -\frac{\partial u}{\partial \mathbf{w}}$$

- O termo d é constante $\rightarrow$ sua derivada é zero.
- Sabemos que:
$$u = \mathbf{w}^T\mathbf{x} - \theta$$
- Como θ é constante, temos:
$$\frac{\partial u}{\partial \mathbf{w}} = \mathbf{x}$$
- Logo:
$$\frac{\partial(d-u)}{\partial \mathbf{w}} = -\mathbf{x}$$
- Substituindo na expressão anterior:
$$\frac{\partial E}{\partial \mathbf{w}} = \frac{1}{2} \cdot 2(d-u) \cdot (-\mathbf{x})$$
- Simplificando:
$$\boxed{\frac{\partial E}{\partial \mathbf{w}} = -(d-u)\mathbf{x}}$$
- Portanto, o gradiente do erro é proporcional ao erro $(d-u)$ e ao vetor de entrada $\mathbf{x}$.